

Tristan Jones  
9/20/25  
CSCI 8110

## Project Assignment 1 Autoencoder

1) Runnable Codes:

Start with loading and normalizing the pixels from 0 to 1

```
import argparse
import os
import time
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

# Keras utilities
from tensorflow.keras.datasets import cifar10
from tensorflow.keras import layers, models, callbacks
```

```
# Load CIFAR-10 images and normalize pixels to [0, 1].
# We only need the images (x) for an autoencoder, not the labels (y).
(x_train, _), (x_test, _) = cifar10.load_data()
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
```

Build Autoencoder creates a Keras functional model for a simple convolutional autoencoder.

```
def build_autoencoder(input_shape=(32, 32, 3)):
    # Input tensor for images
    input_img = layers.Input(shape=input_shape)

    # -----
    # Encoder: reduce spatial dimensions and increase channels
    # Conv2D -> MaxPool reduces HxW while learning features
    # -----
    x = layers.Conv2D(32, (3, 3), activation="relu", padding="same")(input_img)
    x = layers.MaxPooling2D((2, 2), padding="same")(x)
    x = layers.Conv2D(64, (3, 3), activation="relu", padding="same")(x)
    encoded = layers.MaxPooling2D((2, 2), padding="same")(x)

    # -----
    # Decoder: upsample back to original image size
    # UpSampling2D reverses pooling; Conv2D refines reconstructed pixels
    # Final layer uses `sigmoid` to produce outputs in [0, 1]
    # Additional Note: Sigmoid function could cause issues with mixed precision training i.e. rounding issues
    # -----
    x = layers.Conv2D(64, (3, 3), activation="relu", padding="same")(encoded)
    x = layers.UpSampling2D((2, 2))(x)
    x = layers.Conv2D(32, (3, 3), activation="relu", padding="same")(x)
    x = layers.UpSampling2D((2, 2))(x)
    decoded = layers.Conv2D(3, (3, 3), activation="sigmoid", padding="same")(x)

    autoencoder = models.Model(input_img, decoded)
    return autoencoder
```

Get datasets creates a random training/testing dataset that are a shuffled subset of the overall CIFAR-10 Dataset.

```
def get_datasets(x_train, x_test, batch_size=128, quick=False):
    # Optionally use a small subset for quick testing
    if quick:
        x_train = x_train[:1024]
        x_test = x_test[:256]

    # Create tf.data pipelines. For autoencoders inputs == targets.
    train_ds = tf.data.Dataset.from_tensor_slices((x_train, x_train))
    # Shuffle large buffer for randomness, batch and prefetch for perf
    train_ds = train_ds.shuffle(10000).batch(batch_size).prefetch(tf.data.AUTOTUNE)

    val_ds = tf.data.Dataset.from_tensor_slices((x_test, x_test))
    val_ds = val_ds.batch(batch_size).prefetch(tf.data.AUTOTUNE)

    return train_ds, val_ds
```

Plot reconstructions show the first n images, comparing the original with the reconstructed version from the model.

```
def plot_reconstructions(model, x_test, n=6, save_path=None):
    # Reconstruct the first `n` images and show original vs reconstructed
    decoded_imgs = model.predict(x_test[:n])
    plt.figure(figsize=(12, 4))
    for i in range(n):
        # Original image
        ax = plt.subplot(2, n, i + 1)
        plt.imshow(x_test[i])
        plt.axis("off")

        # Reconstructed image
        ax = plt.subplot(2, n, i + 1 + n)
        plt.imshow(decoded_imgs[i])
        plt.axis("off")

    # Save or display the figure
    if save_path:
        plt.savefig(save_path, bbox_inches="tight")
    else:
        plt.show()
```

The rest of the program helps to call the other functions as mentioned in the code. I used GPT 5/ Co-Pilot to assist me in creating this program.

2) Plot a few (3 to 6) examples of input images and their corresponding outputs:

This is the output with no additions or alterations to the functions using both MAE and Adam. I will go further into this in the discussion here, but essentially, it does an excellent job of capturing those bigger details, but struggles with the finer details.

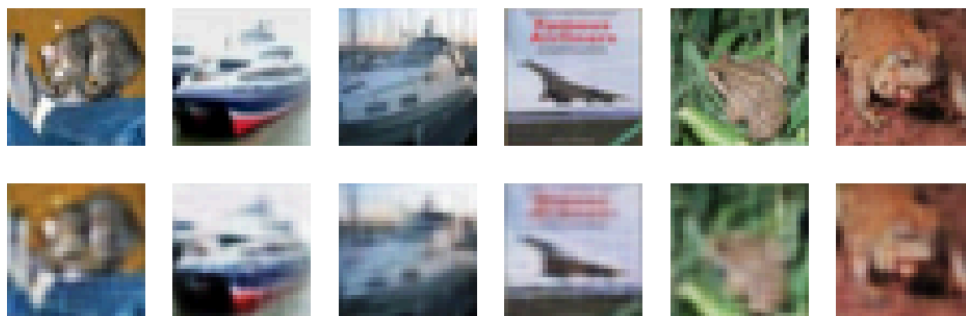


Figure 1: 32/64 filter counts

### 3) Discussion:

The goal of this project was to design and implement a conventional autoencoder to compress and reconstruct images from the CIFAR-10 dataset. This dataset consists of 60,000 small 32x32 RGB images from across 10 different object categories, which makes this a pretty good test case for exploring this kind of image reconstruction since the images are so small. From the instructions, the model was trained using the Adam optimizer with mean absolute error (MAE) as the loss function. The network here uses a convolutional encoder to reduce the spatial dimensions with Conv2d() and MaxPooling2D(), as well as a symmetric decoder to reconstruct the images with Conv2d() and UpSampling2D(). The objective here is to learn a compressed representation that preserves as much visual detail as possible.

For the optimizer we used Adam and for the loss function we used mean absolute error (MAE). Alternative loss functions include mean squared error (MSE) or root mean squared error (RMSE). The key difference is that MSE penalizes larger errors more heavily due to squaring, while MAE grows linearly with the magnitude of the error. As a result, MAE is less sensitive to outliers in the dataset. Given the nature of the CIFAR-10 dataset, where pixel values can vary widely and small outlier regions may occur, MAE provided a more balanced error measure. In practice, this often leads to slightly sharper reconstructions compared to MSE, which tends to produce smoother outputs.

From looking at the results, the color tone and overall silhouette of the objects came out pretty good. The autoencoder did a nice job of keeping the relative shape and some of the key lines. For example, on the boats, you can still make out the structure pretty clearly, and the airplanes also stayed fairly recognizable. Where it struggled was with the finer details, especially on animals. The fur on the cats and the skin on the frogs basically got smoothed out, and small text details, like the writing near the airplane, ended up very blurry and hard to read. Overall, things

like boats and planes looked much easier to recognize in the reconstructions, while animals were tougher since their features rely more on subtle textures.

From the test samples I looked at, the autoencoder got a mean absolute error (MAE) of about 0.033. Scale of 0-1, where lower is better, which means it was doing a decent job on the reconstructions. The structural similarity index (SSIM) was around 0.897, again scale of 0-1, this time where 1 is better. This indicates the outputs matched the originals pretty well, given the small  $32 \times 32$  images. For the peak signal-to-noise ratio (PSNR), I got about 27 dB. Ideally, we'd like 30+ for really high quality, but 27 is still a solid result. Overall, these numbers back up what I saw visually, the model is good at keeping the main shapes and colors, but there's still room to improve if I trained longer or tweaked the architecture.

#### Bonus Section: Extra experimentation

I want to add this section to add some additional experimentation to this. I want to compare Filter counts and additionally compare the difference between MAE and MSE. So for this there were some updates to the code. There were updates to the original functions themselves to enable variability such that we can choose between the different filter counts being 32/64 and 64/128, as well as a variability between MAE and MSE. In addition to this, there were a few new functions added for actually evaluating all of these metrics. The `plot_training_curves()` function just plots the training curves for us given all of our metrics and information. The `save_histort_csv()` function is really useful because it allows us to save the values we get for comparison between the 32/64 vs 64/128 and lastly the `evaluate_metrics` function is just a function that runs all the evaluation metrics essentially. Lastly, there was another file created called `compare_training_curves.py` this file just essentially works to parse through those CSV files that were created and look at the difference between these different metrics.

When looking at the difference between 32/64 filters and 64/128 filters we can see that the pictures definitely are significantly clearer. The boats/planes specifically look like they are almost the same

pictures and for the animal pictures they certainly look better than the 32/64 but still missing some details.

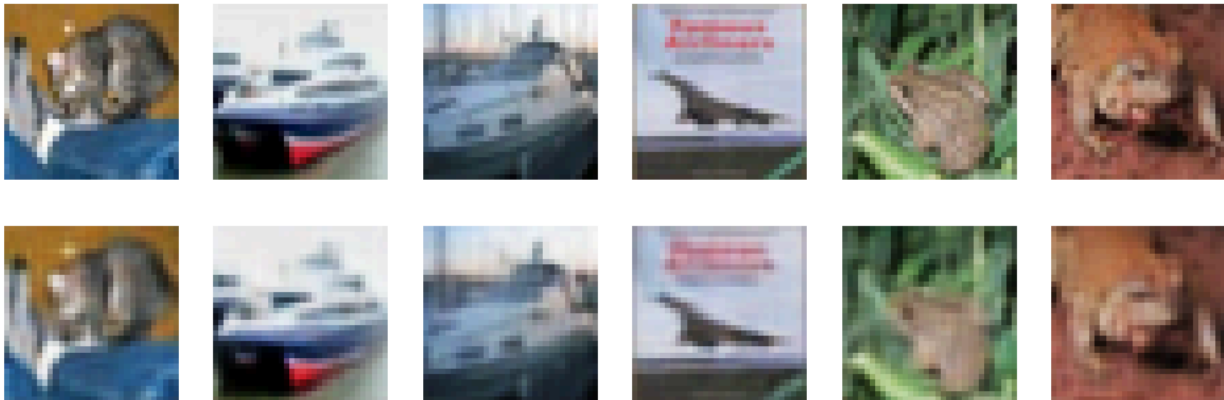
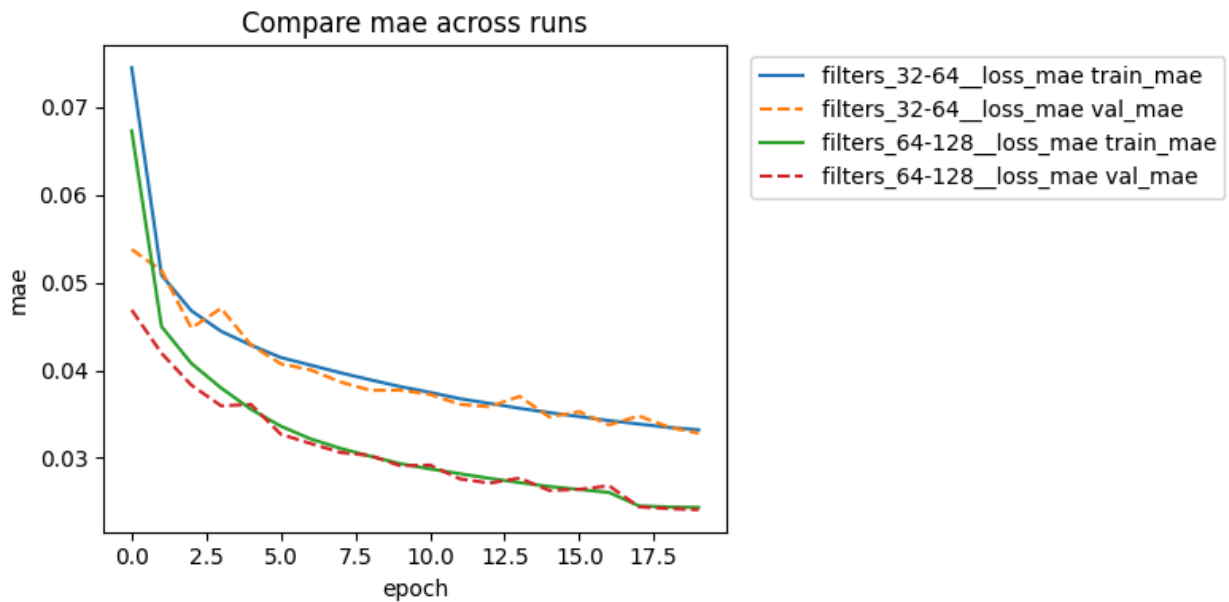


Figure 2: 64/128 filter counts

When looking at the difference in the training curves between these two they are pretty similar with the 64/128 curve ending up just slightly lower at 0.024 as compared to the 0.032 that we get from the 32/64. Additionally our ssim: 0.898 psnr: ~27.09 dB is what we have for our 32/64 and for the 64/128 we get a ssim: 0.940 psnr: ~29.72dB so theres improvements on both fronts here.



For our other test, I wanted to look at the difference between using MSE vs MAE. From the pictures themselves, I would actually say it doesn't seem like there is a huge difference here.

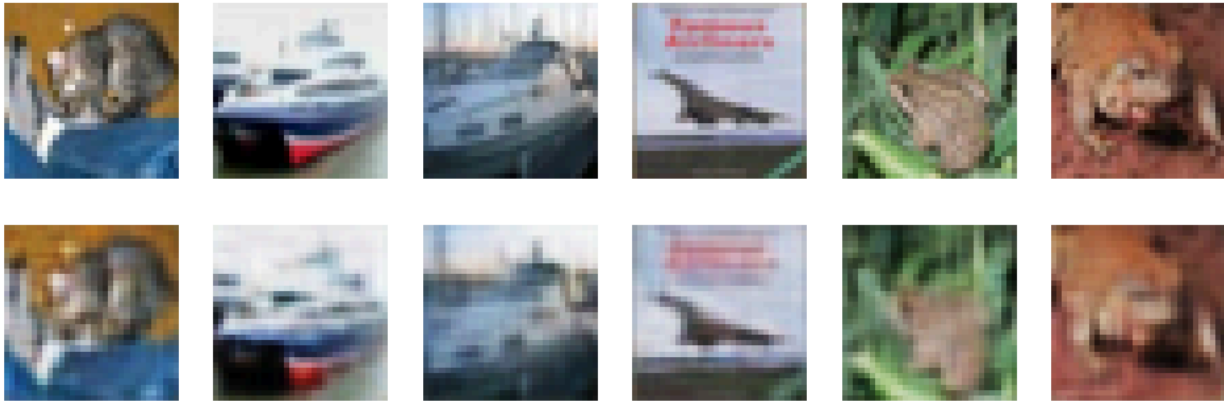


Figure 3: Pictures filters with 32/64 using MSE instead of MAE

I would say that there wasn't really a huge discernable change or difference here from the pictures that we saw originally. From the metrics themselves, it does look like our test loss value of MSE did end up being 0.00207, down from the 0.0328; however, these are 2 different loss functions so kind of hard to compare them in this case exactly. As for the SSIM for MSE it is up to 0.901 from 0.89,8 which is a very small difference. Lastly, the PSNR was  $\sim 27.4$ , up from  $\sim 27.1$  so really not a lot of big changes between these. I think this is kind of the expected result especially after seeing the pictures themselves that there wouldn't be a large change between the two.

Overall, it looks like increasing the filter capacity does yield for better, clearer improvements, while switching the loss function really only showed marginal changes in the picture quality. I think at least for this architecture, the capacity matters more for loss choice. There's certainly other methods and things to test out for this type of work and if I had more time I would love to try and do that but I just don't have that in me for the time being.

**Resources Used:**

- Lecture notes (CSCI 8110, Autoencoders)
- GeeksforGeeks tutorials on autoencoders, extra research
- ChatGPT 5.0 (for help with clarity on writing)
- GitHub Copilot (for code assistance)